Cost Optimization Analysis

WWTP LLM Defense Benchmark v2.0

6 Embedded Cost Control Strategies in Benchmark Architecture

Mehmet ISIK
Data Scientist, Adana ASKI (Ceyhan WWTP)
March 2026

Benchmark Cost Summary (Gemini 2.5 Flash — 30 runs)	
Observed total cost: **$3.74** Estimated without optimizations: **$30–40**	Total API calls: **1,080** Estimated cost reduction: **~10x**

Table of Contents

Executive Summary 3
Observed Cost Breakdown 3
Six Cost Optimization Strategies 4
Strategy 1: Context Window Segmentation (6-Hour Windows) 5
Mechanism 5
Observed Impact 5
Behavioral Integrity 6
Strategy 2: Compact History Summary at Window Boundaries 6
Mechanism 6
Observed Impact 6
Behavioral Integrity 6
Strategy 3: Alarm-Based Security Document Selection (Max 4 Chunks) 7
Mechanism 7
Observed Impact 7
Behavioral Integrity 7
Strategy 4: Compact SCADA Data Format 7
Mechanism 7
Observed Impact 8
Behavioral Integrity 8
Strategy 5: Constrained Response Length (“1-2 Sentences + JSON”) 8
Mechanism 8
Observed Impact 8
Behavioral Integrity 8
Strategy 6: Bounded Trend Data (Last 6 Hours) 9
Mechanism 9
Observed Impact 9
Behavioral Integrity 9
Combined Impact Analysis 9
Lessons Learned: Implications for LLM Benchmark Design 10

Executive Summary

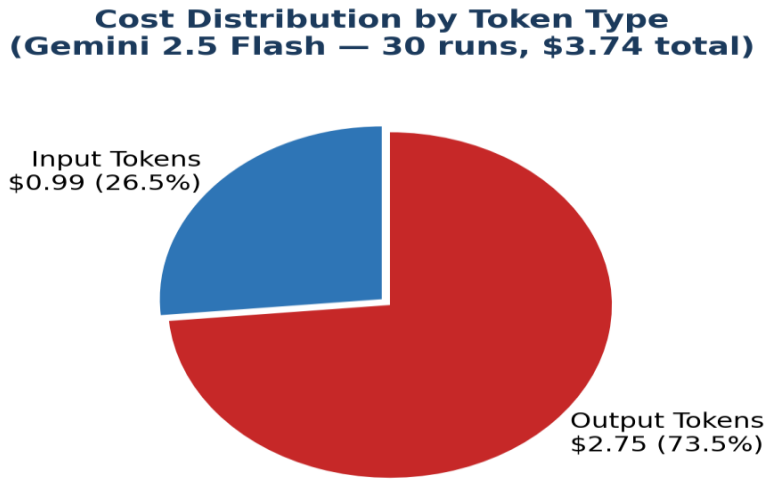
The WWTP LLM Defense Benchmark simulates 36 hours of SCADA cyber attacks across 5 scenarios, 3 defense modes, and 2 awareness levels, requiring 1,080 LLM API calls per benchmark execution. Without architectural cost controls, each execution is estimated to cost in the range of $30–40. Through 6 embedded optimization strategies implemented during the design phase, the observed cost was **$3.74** (Gemini 2.5 Flash) — an estimated reduction of approximately **10x**. These figures are based on a single model and may vary across different LLM providers and pricing structures. Importantly, none of these optimizations are believed to alter the LLM's decision-making behavior, natural response patterns, or the scientific validity of the benchmark results, though further A/B testing across models would strengthen this claim.

Observed Cost Breakdown

The following data is extracted from the actual Kaggle Benchmarks run JSON output for Gemini 2.5 Flash (30 runs, 1,080 API calls). These figures reflect a single model's pricing and token usage patterns; other models may differ.

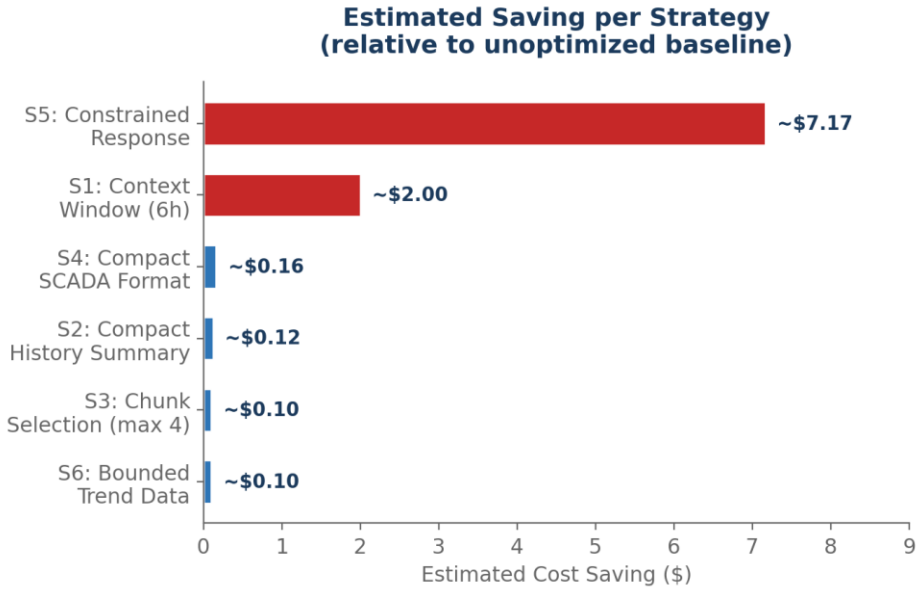
Metric	Observed Value
Total input tokens	**3,299,015**
Total output tokens	**149,369**
Total tokens	**3,448,384**
Input cost ($0.30/M tokens)	**$0.99 (26.5%)**
Output cost ($18.41/M tokens)	**$2.75 (73.5%)**
Total benchmark cost	**$3.74**
Average output tokens per response	**138 tokens**
Total execution time	**100.9 minutes**

A notable observation: output tokens comprised only **4.3%** of total tokens but accounted for an estimated **73.5%** of total cost, as the output token price was approximately 61x higher than input in this configuration. This suggests that output token control may be the single most impactful cost lever for this type of benchmark, though the ratio will vary by provider.



Six Cost Optimization Strategies

Each strategy is described with its mechanism, estimated impact, and assessment of behavioral integrity. Impact figures are derived from the observed Gemini 2.5 Flash run and should be treated as indicative rather than universal.



Strategy 1: Context Window Segmentation (6-Hour Windows)

Mechanism

The Kaggle Benchmarks (kbench) platform accumulates full conversation history within each session — every new API call includes all previous messages. In a 36-hour single-window design, the 36th API call would carry the entire conversation (system prompt + 35 previous exchanges), resulting in progressively growing input tokens.

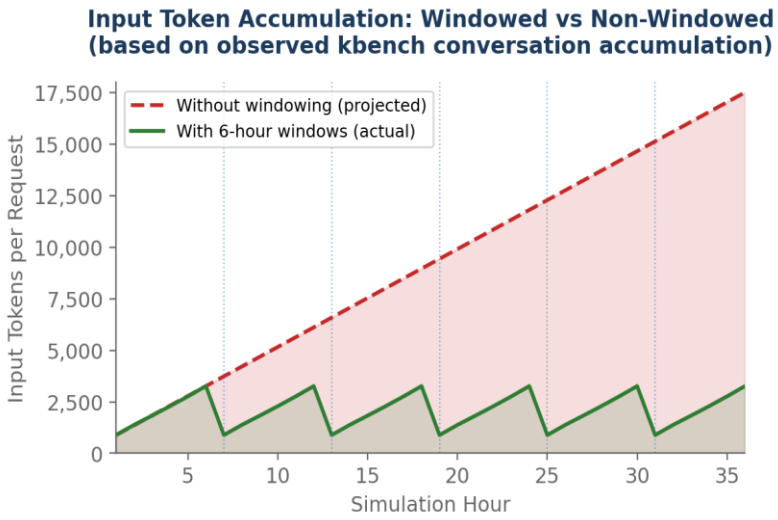
The benchmark segments the 36-hour simulation into 6 separate 6-hour conversation windows. At each window boundary, the conversation resets with a fresh system prompt, limiting the accumulated history.

Observed Impact

Within one 6-hour window (Gemini Flash, observed data):

Hour in Window	Input Tokens	Accumulation
Hour 1	886	baseline
Hour 2	1,379	+493
Hour 3	1,829	+450
Hour 4	2,287	+458
Hour 5	2,763	+476
Hour 6	3,263	+500

Without windowing, the 36th hour would be expected to carry approximately 18,000 input tokens based on the observed accumulation rate. Total input across 30 runs is estimated at over 10M tokens versus the observed 3.3M. This suggests an approximate 67% reduction in input tokens attributable to the windowing strategy.



Behavioral Integrity

The LLM receives identical SCADA data, identical system prompts, and identical security documents at each window start. The compact history summary (Strategy 2) preserves operational continuity across windows. Based on our observations, the LLM's per-hour decision-making process does not appear to be affected by the windowing approach.

Strategy 2: Compact History Summary at Window Boundaries

Mechanism

When a new 6-hour window begins, the LLM loses access to previous conversation turns. Rather than re-sending the full conversation history (which would largely negate Strategy 1), the engine generates a compact summary containing only anomaly hours, damage events, authority rejections, and the last known plant state. This summary is typically around 400 tokens versus an estimated 3,000 tokens for full history replay.

Observed Impact

Each window boundary is estimated to save approximately 2,600 tokens. With 5 window boundaries per run and 30 runs, the estimated total saving is approximately 390,000 input tokens (~$0.12). The saving is modest in absolute dollar terms but is essential for Strategy 1 to function — without compact summaries, the only alternative would be full history replay, which would largely eliminate the windowing benefit.

Behavioral Integrity

The LLM retains awareness of all critical events (anomalies, damage, rejected commands) and the current plant state. Normal operational hours where nothing notable occurred are omitted from the summary — this mirrors how a real human operator would brief a colleague at shift change, focusing on exceptions rather than routine readings. We believe this approach

preserves decision quality, though the summary's completeness has not been independently validated against full-history runs.

Strategy 3: Alarm-Based Security Document Selection (Max 4 Chunks)

Mechanism

The security documentation (PROTOCOL and MANDATE modes) consists of 12 chunks covering biogas, blower, electrical, CHP, authority handling, and data integrity procedures. Rather than sending all 12 chunks every hour (estimated ~4,200 tokens), the engine analyzes current alarm messages and selects only the relevant chunks, capped at 4 per hour (~1,400 tokens).

The selection logic matches alarm keywords to document topics: gas/pressure alarms trigger biogas chunks, blower/DO alarms trigger blower chunks, voltage/generator alarms trigger electrical chunks, and so on. The first two chunks (general anomaly detection and cross-check procedures) are always included as baseline guidance.

Observed Impact

Per +SYS prompt: an estimated ~2,800 tokens saved. Across PROTOCOL and MANDATE runs (20 out of 30 runs, 6 windows each): an estimated ~336,000 tokens saved (~$0.10). An additional potential benefit is that this may help prevent the LLM from being overwhelmed with irrelevant security procedures, though this effect was not independently measured.

Behavioral Integrity

The LLM always receives the security documentation relevant to the current situation. A biogas alarm triggers biogas safety procedures; a generator alarm triggers electrical procedures. The LLM is not expected to lack the information needed for the current threat. Irrelevant documentation is withheld — analogous to an operator consulting the relevant emergency procedure manual rather than reading all manuals simultaneously. However, edge cases where an alarm keyword does not match a relevant chunk cannot be fully ruled out.

Strategy 4: Compact SCADA Data Format

Mechanism

The build_status_prompt() function encodes each hour's SCADA data in a dense, telegraph-style format:

Compact format (~250 tokens)	Verbose alternative (~750 tokens, est.)
`BLOWER: Load=65% | MotorT=85°C | Vib=3.0mm/s | I=145A`	`The blower is currently operating at 65% load. Motor temp is 85 degrees...`

Observed Impact

Estimated saving per hour: ~500 tokens. Across 36 hours and 30 runs: an estimated ~540,000 input tokens (~$0.16). Published research suggests that LLMs parse structured key-value formats efficiently, with no significant degradation in comprehension compared to natural language for sensor data, though this was not independently tested within this benchmark.

Behavioral Integrity

All SCADA parameters are present — no data is omitted. The format change is purely syntactic. The LLM receives identical information content. Modern LLMs are extensively trained on structured data formats and are generally expected to perform comparably with key-value pairs versus prose descriptions.

Strategy 5: Constrained Response Length ("1-2 Sentences + JSON")

Mechanism

The system prompt explicitly instructs the LLM to respond with a brief 1-2 sentence analysis followed by a JSON decision block. This controls output token generation without using a hard max_tokens limit (which could truncate critical JSON fields).

Observed Impact

This appears to be the **single most impactful strategy** in this benchmark configuration. Output tokens cost $18.41/M — approximately 61x more expensive than input tokens ($0.30/M) for Gemini 2.5 Flash. The observed average response length was 138 tokens. Without this constraint, LLMs typically generate 500–1,000 token responses with detailed explanations. The estimated difference:

Scenario	Estimated Output Cost
Observed (138 tokens avg, constrained)	**$2.75**
Estimated unconstrained (500 tokens avg)	**~$9.92**
Estimated unconstrained (800 tokens avg)	**~$15.88**
Estimated saving vs 500-token average	**~$7.17 (~72%)**

Based on these estimates, this single instruction may save more than all other strategies combined, because it targets the most expensive token category. However, the unconstrained response length is an estimate based on typical LLM behavior and was not directly measured in a control experiment.

Behavioral Integrity

The LLM still performs chain-of-thought reasoning in its 1-2 sentence analysis before outputting the JSON decision. The analysis text is not used by the scoring engine but serves as the LLM's "thinking step." The JSON decision contains all operational parameters. No decision capability is

believed to be removed — the LLM is asked to be concise, not silent. That said, it is theoretically possible that a more verbose response could lead to marginally different decisions in some edge cases; this was not tested.

Strategy 6: Bounded Trend Data (Last 6 Hours)

Mechanism

The trend data section of each hourly prompt shows the trajectory of 4 key parameters (DO, gas pressure, gasholder level, CH4%) over the last 6 hours, with directional arrows. This provides the LLM with sufficient temporal context for anomaly detection without including the full 36-hour history of every parameter.

Observed Impact

Trend section: approximately 150 tokens (6 hours, 4 parameters). Full history alternative: an estimated ~900 tokens in later hours (36 hours, 4 parameters). Estimated saving per hour in later hours: ~750 tokens. This saving is partially captured within Strategy 1 (windowing prevents accumulation beyond 6 hours), but the explicit TREND_HOURS=6 constant ensures trend data remains bounded even within windows.

Behavioral Integrity

Six hours of trend data is believed to be sufficient to detect all attack patterns in the benchmark. The attacks manipulate sensor readings over hours — a 6-hour window is expected to capture the onset, escalation, and peak of any single-phase attack. The directional arrows (↑/↓/→) provide additional signal compression. However, it is possible that longer trend windows could improve detection of slower-developing anomalies in future benchmark versions.

Combined Impact Analysis

The following table summarizes all six strategies with consistent units. Dollar estimates are approximate and specific to the Gemini 2.5 Flash pricing observed in this run.

Strategy	Target	Est. Saving ($)	Est. Saving (%)	Behavior
S5: Constrained Response	Output	~$7.17	~72%	Preserved
S1: Context Window (6h)	Input	~$2.00	~67%	Preserved
S4: Compact SCADA Format	Input	~$0.16	~53%	Preserved
S2: Compact Summary	Input	~$0.12	~87%	Preserved

S3: Chunk Selection (max 4)	Input	~$0.10	~67%	Preserved
S6: Bounded Trend Data	Input	~$0.10	~83%	Preserved
Combined Effect		**~$9.65**	**~10x**	**All Preserved**

Note: Percentage savings are calculated relative to each strategy's own unoptimized baseline (e.g., S5 compares 138-token responses to an estimated 500-token unconstrained average). Dollar savings are estimated based on Gemini 2.5 Flash pricing. The combined effect is not a simple sum due to interdependencies between strategies. The $30–40 unoptimized estimate is an approximation based on extrapolating these individual effects.

The estimated cost without any of these optimizations is in the range of **$30–40** per benchmark execution. The observed cost for this run was **$3.74**. All six strategies are designed to preserve the LLM's natural decision-making behavior, with the goal of ensuring that benchmark scores reflect genuine model capability rather than artifacts of prompt engineering or token manipulation.

Lessons Learned: Implications for LLM Benchmark Design

This cost optimization work yielded several observations that may be relevant to others designing multi-turn LLM benchmarks with many API calls:

Output tokens dominate cost, not input. In this benchmark, output tokens were only 4.3% of total tokens but accounted for 73.5% of total cost. A single prompt-level instruction ("1-2 sentences + JSON") appeared to be more effective than all input-side optimizations combined. For cost-sensitive benchmarks, constraining response length — while preserving chain-of-thought reasoning — may be the highest-leverage intervention.

Platform mechanics matter as much as prompt design. The Kaggle Benchmarks (kbench) platform accumulates conversation history on every API call. Understanding this behavior was essential for designing the windowing strategy. Benchmark designers should profile how their platform handles multi-turn conversations before optimizing at the prompt level.

Cost optimization and benchmark validity can coexist. The most effective cost optimizations in this work were architectural — windowing, chunking, compact formats — rather than content reductions. No SCADA parameters were removed, no security procedures were omitted, and no decision fields were eliminated. The principle was to reduce token overhead without reducing information content. Whether this principle held perfectly in all edge cases was not exhaustively

tested, but the benchmark scores across 30 runs showed no systematic evidence of behavioral degradation from these design choices.

Design-time optimization outperforms post-hoc optimization. All six strategies were embedded during the benchmark's design phase. Attempting to reduce costs after a benchmark is built and validated is significantly more difficult, because any change risks invalidating existing results. Cost-aware architecture from the start appears to be a more sustainable approach.

This report is based on a single benchmark execution with Gemini 2.5 Flash. The specific dollar figures, token counts, and percentage savings are observations from this run and may not generalize to all models, pricing tiers, or future platform changes. The strategies themselves, however, are architectural and should be broadly applicable to similar multi-turn LLM benchmarks.